

```

# =====
# 9. Forecasting
# =====

# -----
# Function
# -----

load_test_data <- function(file_path) {
  df_test <- read.csv(file_path, stringsAsFactors = FALSE)

  required_cols <- c("date", "hpi")
  missing_cols <- setdiff(required_cols, names(df_test))

  if (length(missing_cols) > 0) {
    stop(paste("Missing required columns:", paste(missing_cols, collapse = ",
")))
  }

  df_test$date <- as.Date(df_test$date)
  df_test <- df_test[order(df_test$date), ]

  if (!"log_hpi" %in% names(df_test)) {
    df_test$log_hpi <- log(df_test$hpi)
  }

  df_test <- add_dummy_variables(df_test)

  return(df_test)
}

get_forecast_horizon <- function(df_test) {
  h <- nrow(df_test)
  return(h)
}

make_test_xreg <- function(df_test, model_name) {
  if (grepl("post-2002", model_name)) {
    xreg_test <- as.matrix(df_test[, c("post2002", "crisis"), drop = FALSE])
  } else if (grepl("crisis dummy", model_name)) {
    xreg_test <- as.matrix(df_test[, c("crisis"), drop = FALSE])
  } else {
    xreg_test <- NULL
  }

  return(xreg_test)
}

is_dummy_model <- function(model_name) {
  grepl("^ARIMAX", model_name) || grepl("^SARIMAX", model_name)
}

forecast_level_model <- function(fit, h) {
  fc <- forecast::forecast(

```

```

    fit,
    h = h,
    level = c(80, 95)
  )

forecast_df <- data.frame(
  Forecast_log = as.numeric(fc$mean),
  Lower_80_log = as.numeric(fc$lower[, "80%"]),
  Upper_80_log = as.numeric(fc$upper[, "80%"]),
  Lower_95_log = as.numeric(fc$lower[, "95%"]),
  Upper_95_log = as.numeric(fc$upper[, "95%"])
)

forecast_df$Forecast_HPI <- exp(forecast_df$Forecast_log)
forecast_df$Lower_80_HPI <- exp(forecast_df$Lower_80_log)
forecast_df$Upper_80_HPI <- exp(forecast_df$Upper_80_log)
forecast_df$Lower_95_HPI <- exp(forecast_df$Lower_95_log)
forecast_df$Upper_95_HPI <- exp(forecast_df$Upper_95_log)

return(forecast_df)
}

forecast_dummy_model <- function(fit, h, xreg_test, last_train_log_hpi) {
  fc <- forecast::forecast(
    fit,
    h = h,
    xreg = xreg_test,
    level = c(80, 95)
  )

  # The dummy models forecast diff(log(HPI)).
  # Convert forecasted monthly growth rates back to log(HPI)
  # by cumulative summation.
  forecast_diff_log <- as.numeric(fc$mean)
  lower_80_diff_log <- as.numeric(fc$lower[, "80%"])
  upper_80_diff_log <- as.numeric(fc$upper[, "80%"])
  lower_95_diff_log <- as.numeric(fc$lower[, "95%"])
  upper_95_diff_log <- as.numeric(fc$upper[, "95%"])

  forecast_log <- last_train_log_hpi + cumsum(forecast_diff_log)
  lower_80_log <- last_train_log_hpi + cumsum(lower_80_diff_log)
  upper_80_log <- last_train_log_hpi + cumsum(upper_80_diff_log)
  lower_95_log <- last_train_log_hpi + cumsum(lower_95_diff_log)
  upper_95_log <- last_train_log_hpi + cumsum(upper_95_diff_log)

  forecast_df <- data.frame(
    Forecast_log = forecast_log,
    Lower_80_log = lower_80_log,
    Upper_80_log = upper_80_log,
    Lower_95_log = lower_95_log,
    Upper_95_log = upper_95_log
  )

  forecast_df$Forecast_HPI <- exp(forecast_df$Forecast_log)
  forecast_df$Lower_80_HPI <- exp(forecast_df$Lower_80_log)

```

```

forecast_df$Upper_80_HPI <- exp(forecast_df$Upper_80_log)
forecast_df$Lower_95_HPI <- exp(forecast_df$Lower_95_log)
forecast_df$Upper_95_HPI <- exp(forecast_df$Upper_95_log)

return(forecast_df)
}

calculate_forecast_accuracy <- function(actual, forecast) {
  error <- actual - forecast

  rmse <- sqrt(mean(error^2))
  mae <- mean(abs(error))
  mape <- mean(abs(error / actual)) * 100

  data.frame(
    RMSE = rmse,
    MAE = mae,
    MAPE = mape,
    row.names = NULL
  )
}

forecast_single_model <- function(model_name, fit, df_test, last_train_log_hpi)
{
  h <- get_forecast_horizon(df_test)

  if (is_dummy_model(model_name)) {
    xreg_test <- make_test_xreg(df_test, model_name)

    forecast_df <- forecast_dummy_model(
      fit = fit,
      h = h,
      xreg_test = xreg_test,
      last_train_log_hpi = last_train_log_hpi
    )
  } else {
    forecast_df <- forecast_level_model(
      fit = fit,
      h = h
    )
  }

  forecast_df$Date <- df_test$date
  forecast_df$Actual_HPI <- df_test$hpi
  forecast_df$Model <- model_name

  accuracy <- calculate_forecast_accuracy(
    actual = forecast_df$Actual_HPI,
    forecast = forecast_df$Forecast_HPI
  )

  accuracy$Model <- model_name
  accuracy <- accuracy[, c("Model", "RMSE", "MAE", "MAPE")]

  output <- list(

```

```

    forecast_df = forecast_df,
    accuracy = accuracy
  )

  return(output)
}

forecast_selected_models <- function(results, diagnostics_results,
test_file_path) {
  df_test <- load_test_data(test_file_path)

  selected_model_names <- diagnostics_results$selected_model_names
  models <- results$models

  last_train_log_hpi <- tail(results$data$log_hpi, 1)

  forecast_outputs <- lapply(selected_model_names, function(model_name) {
    forecast_single_model(
      model_name = model_name,
      fit = models[[model_name]],
      df_test = df_test,
      last_train_log_hpi = last_train_log_hpi
    )
  })

  names(forecast_outputs) <- selected_model_names

  forecast_table <- do.call(
    rbind,
    lapply(forecast_outputs, function(x) x$forecast_df)
  )

  accuracy_table <- do.call(
    rbind,
    lapply(forecast_outputs, function(x) x$accuracy)
  )

  accuracy_table <- accuracy_table[order(accuracy_table$RMSE), ]

  cat("\n===== Forecast Accuracy Table =====\n")
  print(accuracy_table, row.names = FALSE, digits = 5)

  output <- list(
    df_test = df_test,
    selected_model_names = selected_model_names,
    forecast_outputs = forecast_outputs,
    forecast_table = forecast_table,
    accuracy_table = accuracy_table
  )

  return(output)
}

# -----
# Visualization helpers

```

```

# -----

add_english_month_axis <- function(dates, by = 2, show_year = FALSE) {
  tick_index <- seq(1, length(dates), by = by)

  if (show_year) {
    tick_labels <- format(dates[tick_index], "%b %Y")
  } else {
    tick_labels <- format(dates[tick_index], "%b")
  }

  axis(
    side = 1,
    at = dates[tick_index],
    labels = tick_labels
  )
}

plot_forecast_with_intervals <- function(forecast_df, model_name) {
  plot(
    forecast_df$Date,
    forecast_df$Actual_HPI,
    type = "l",
    lwd = 2,
    xaxt = "n",
    ylim = range(
      forecast_df$Lower_95_HPI,
      forecast_df$Upper_95_HPI,
      forecast_df$Actual_HPI,
      na.rm = TRUE
    ),
    xlab = "Date",
    ylab = "HPI",
    main = paste("12-Month HPI Forecast:", model_name)
  )

  add_english_month_axis(
    dates = forecast_df$Date,
    by = 2,
    show_year = FALSE
  )

  lines(
    forecast_df$Date,
    forecast_df$Forecast_HPI,
    lwd = 2,
    lty = 2
  )

  lines(
    forecast_df$Date,
    forecast_df$Lower_95_HPI,
    lty = 3
  )
}

```

```

lines(
  forecast_df$Date,
  forecast_df$Upper_95_HPI,
  lty = 3
)

lines(
  forecast_df$Date,
  forecast_df$Lower_80_HPI,
  lty = 4
)

lines(
  forecast_df$Date,
  forecast_df$Upper_80_HPI,
  lty = 4
)

legend(
  "topleft",
  legend = c("Actual HPI", "Forecast HPI", "95% PI", "80% PI"),
  lty = c(1, 2, 3, 4),
  lwd = c(2, 2, 1, 1),
  bty = "n"
)
}

plot_all_selected_forecasts <- function(forecasting_results) {
  selected_model_names <- forecasting_results$selected_model_names

  for (model_name in selected_model_names) {
    cat("\nPlotting forecast for:", model_name, "\n")

    forecast_df <-
forecasting_results$forecast_outputs[[model_name]]$forecast_df

    plot_forecast_with_intervals(
      forecast_df = forecast_df,
      model_name = model_name
    )

    readline(prompt = "Press [Enter] to continue to the next forecast plot...")
  }
}

plot_forecast_comparison <- function(forecasting_results) {
  forecast_table <- forecasting_results$forecast_table
  df_test <- forecasting_results$df_test
  selected_model_names <- forecasting_results$selected_model_names

  plot(
    df_test$date,
    df_test$hpi,
    type = "l",
    lwd = 2,

```

```

    xaxt = "n",
    ylim = range(
      forecast_table$Forecast_HPI,
      df_test$hpi,
      na.rm = TRUE
    ),
    xlab = "Date",
    ylab = "HPI",
    main = "Forecast Comparison for Retained Models"
  )

  add_english_month_axis(
    dates = df_test$date,
    by = 2,
    show_year = FALSE
  )

  for (model_name in selected_model_names) {
    model_forecast <- forecast_table[forecast_table$Model == model_name, ]

    lines(
      model_forecast$Date,
      model_forecast$Forecast_HPI,
      lty = 2
    )
  }

  legend(
    "topleft",
    legend = c("Actual HPI", selected_model_names),
    lty = c(1, rep(2, length(selected_model_names))),
    lwd = c(2, rep(1, length(selected_model_names))),
    bty = "n",
    cex = 0.7
  )
}

run_forecasting_analysis <- function(results, diagnostics_results,
test_file_path) {
  forecasting_results <- forecast_selected_models(
    results = results,
    diagnostics_results = diagnostics_results,
    test_file_path = test_file_path
  )

  plot_forecast_comparison(forecasting_results)
  readline(prompt = "Press [Enter] to continue to individual forecast
plots...")

  plot_all_selected_forecasts(forecasting_results)

  return(forecasting_results)
}

```

```
# =====  
# Main  
# =====  
  
forecasting_results <- run_forecasting_analysis(  
  results = results,  
  diagnostics_results = diagnostics_results,  
  test_file_path = "C:/Users/use/Desktop/NCCU_STAT/Time  
Series/final/data/processed/monthly_housing_test.csv"  
)
```